

CCAF Day 1

Agentic Architecture & Orchestration

Exam Domain: Agentic Architecture & Orchestration | **Blueprint Weight:** 27% | **Status:** First-pass learning complete

These notes capture everything learned so far in Day 1, including concepts, examples, architectural reasoning, and key exam principles. Scenario practice and the final Day-1 assessment are still pending.

1. What is an AI Agent?

Definition: An AI agent is an AI system that works toward a goal by deciding what actions to take and using available tools or capabilities to accomplish that goal.

```
Goal: Find why today's data pipeline failed
↓
Check pipeline status
↓
FAILED
↓
Check logs
↓
Schema mismatch
↓
Compare schemas
↓
customer_id: INTEGER → STRING
↓
Root cause identified
↓
Return result
```

Core idea: An agent does not merely generate an answer. It can work toward a goal by choosing actions, observing results, and deciding what to do next.

2. Agent vs Workflow

Workflow: A predefined sequence of actions where the developer/application primarily defines the path.

```
Get Metadata
↓
Validate File
↓
Transform
↓
Load Table
↓
Send Notification
```

Agent: Receives a goal and can dynamically decide the next action based on what it discovers.

```
Goal
↓
Decide
↓
Act
↓
Observe
↓
Decide
```

↓
Act
↓
...

Exam thinking: Ask “Who determines the sequence of actions?” If the path is largely predefined, think workflow. If the agent can dynamically choose the next action from observations, think agentic behavior.

Nuance: An AI-powered workflow can contain an LLM and still be a workflow. Do not memorize “workflow = no AI.”

3. Agent Loop

A basic agent loop can be understood as **Goal → Decide → Act → Observe → Decide → ... → Stop**.

```
Goal
↓
Decide
↓
Act
↓
Observe
↓
Goal complete?
■ ■
YES NO
↓ ↓
STOP Continue
```

The agent should not blindly execute every available tool. It continually considers: **“Have I achieved the goal? If not, what should I do next?”**

4. Tools / Capabilities

A **tool** is a capability provided to an AI agent that allows it to interact with something outside the model.

```
check_pipeline_status()
get_pipeline_logs()
compare_schemas()
query_metadata()
read_source_file()
send_email()
```

A tool is not necessarily a database table. It may internally query a database, call an API, read a file, search a system, execute code, or interact with another service.

Developer provides the capabilities. The agent decides which relevant capability to use based on the goal, context, observations, and constraints.

5. Autonomy

Autonomy is the degree of freedom an agent has to decide what actions or tools to use to accomplish a goal.

Low autonomy:

```
Check status
↓
If failed → Check logs
↓
If schema issue → Compare schemas
```

Higher autonomy:

Goal: Investigate today's pipeline failure

Tools:

- Check status
- Get logs
- Compare schemas
- Query metadata
- Check source

Constraints:

- Cannot modify production
- Cannot delete data

Key definition: Autonomy = freedom to decide, within constraints.

More autonomy is not automatically better. High-impact actions may require restrictions, approval, or human oversight.

6. Agent Roles & Specialization

A role defines the responsibility or focus of an agent. Specialized agents can have different instructions, tools, responsibilities, and constraints.

Pipeline Agent

→ Investigates pipeline execution problems

Data Agent

→ Investigates data and schema problems

Infrastructure Agent

→ Investigates credentials, access, and infrastructure

Important: “Role” does not necessarily mean a separate agent. A role describes responsibility/behavior; multiple roles can be implemented in different ways.

7. Handoff

A **handoff** occurs when responsibility for the next part of a task is transferred from one agent to another specialized agent.

Pipeline Agent

↓

Finds schema mismatch

↓

HANDOFF

↓

Data Agent

↓

Compare schemas

↓

Identify changed column

Example: if the pipeline fails because credentials expired, the issue should go to an Infrastructure/Access specialist rather than a Data Agent.

Principle: The next agent should be selected based on the responsibility required by the current problem.

8. Why Use Multiple Specialized Agents?

A single agent with many tools can become harder to manage for complex systems. Specialization can provide separation of responsibilities, better tool organization, easier maintenance and debugging, and better handling of complex tasks.

```

Manager
/ | ↓ ↓ ↓
Pipeline Data Infra
Agent Agent Agent

```

Trade-offs: More agents introduce coordination, communication, latency, cost, failure points, and architectural complexity.

Architecture principle: Use the simplest architecture that can reliably accomplish the goal.

9. Supervisor / Manager Pattern

A Supervisor/Manager Agent coordinates specialized agents. It can route work, collect results, coordinate execution, determine when the overall task is complete, and help assemble the final response.

```

Manager Agent
↓
Investigate failure
↓
Pipeline Agent
↓
What is the issue?
■ ■
Schema issue Credential issue
↓ ↓
Data Agent Infrastructure Agent

```

Supervisor: coordinates which agent handles the next part.

Handoff: transfers responsibility from one agent to another.

Supervisor and handoff patterns can be combined in the same architecture.

10. Single-Agent Architecture

A single agent can have several relevant tools and handle the complete task.

Agent

 ↓ ↓ ↓
 Pipeline SQL Schema
 Tools Tools Tools

Prefer a single agent when the task is relatively simple, one agent can handle it reliably, specialization is unnecessary, and multi-agent coordination would add unnecessary complexity.

11. Multi-Agent Architecture

A multi-agent architecture uses multiple agents with different responsibilities.

```

Manager
/ | ↓ ↓ ↓
Pipeline Data Infra
Agent Agent Agent

```

It may help for complex, naturally divisible tasks that require different expertise or tools. However, multi-agent architecture is not automatically better.

12. Task Decomposition

Task decomposition means breaking a large goal into smaller tasks.

Main Goal

-
- Analyze sales
- Identify top 5 products
- Compare regions
- Explain results
- Create summary

Task decomposition ≠ multi-agent architecture. Decomposition answers “What smaller tasks make up this goal?” It does not automatically decide which agent performs them.

13. Sequential vs Parallel Execution

Sequential: Use when tasks have dependencies.

Find customer ID
↓
Get customer's orders

Parallel: Consider when tasks are sufficiently independent.

■ → Research Company A
Goal ■■■ → Research Company B
■ → Research Company C

Simple rule: Independent tasks → consider parallel execution. Dependent tasks → execute in the required sequence.

Parallel execution can reduce latency when tasks can safely run concurrently.

14. Planning

Planning is determining how to accomplish a goal through a useful sequence of actions.

Goal: Find why pipeline failed

Plan:

1. Check status
2. If failed → get logs
3. Identify failure category
4. Investigate specific cause
5. Report root cause

Task decomposition breaks the goal into smaller tasks. **Planning** determines how/when those tasks should be accomplished.

Plans can be dynamic. If observations change the situation, the agent can re-plan.

Not every task needs an elaborate plan. Simple tasks can often be executed directly.

15. State

State is information about the current situation and progress of an agent's task.

Goal:

Find why pipeline failed

Pipeline status:

FAILED

Log result:

Schema mismatch

Next investigation:
Compare schemas

State helps the agent understand: **“Where am I in the task, and what do I already know?”**

State can include the goal, current task, previous actions, tool results, intermediate findings, decisions, and progress.

State does not necessarily mean permanent memory. A task's state may exist only while the task is running.

16. Human-in-the-Loop (HITL)

Human-in-the-loop means intentionally involving a human in an agent's decision or action process.

```
Agent
↓
Decides: "Drop production table"
↓
High-impact action
↓
Human approval
↓
Approve?
■ ■
YES NO
↓ ↓
Execute Stop/revise
```

HITL is especially useful for destructive, irreversible, financial, security-sensitive, production, or compliance-sensitive actions.

Not every action requires approval. Low-risk actions can remain automated; high-impact actions may require human approval.

17. Failure Handling & Retries

Tools and external services can fail. The agent/system should distinguish between failures that may be transient and failures that are persistent or structural.

```
get_pipeline_logs()
↓
TIMEOUT
↓
Retry
↓
SUCCESS
↓
Continue
```

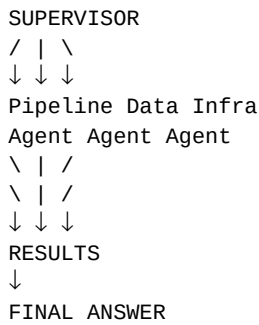
A timeout may be transient. A permission-denied error may not be fixed by repeated retries.

```
Attempt 1 → fail
Attempt 2 → fail
Attempt 3 → fail
↓
Stop / escalate
```

Important: Not every failure should be retried. Consider whether the failure is transient, whether retrying is safe, and whether a retry limit exists.

STOP Re-plan / Act

For complex systems, a supervisor may coordinate specialized agents and their results.



21. Key Definitions for Revision

Concept	Simple meaning
Agent	AI system that works toward a goal by deciding actions.
Tool	Capability available to an agent.
Agent loop	Decide → Act → Observe → Decide → Stop.
Workflow	Predefined sequence of actions.
Autonomy	Freedom to decide within constraints.
Role	Responsibility/behavior assigned to an agent.
Specialization	Agents focus on specific responsibilities.
Handoff	Transfer of responsibility between agents.
Supervisor	Coordinates specialized agents.
Single-agent	One agent handles the task.
Multi-agent	Multiple specialized agents collaborate.
Task decomposition	Break a large goal into smaller tasks.
Sequential	Tasks execute in dependency order.
Parallel	Independent tasks execute concurrently.
Planning	Determine how to accomplish the goal.
State	Current task information and progress.
HITL	Human approval/participation at selected points.
Retry	Repeat an operation after a suitable failure.
Idempotency	Safety of repeating an operation without unwanted duplicate effects.
Orchestration	Coordination of agents, tools, tasks, and execution.

22. Architecture Principles to Remember

1. Use the simplest architecture that can reliably accomplish the goal.
2. More autonomy is not automatically better.
3. More agents are not automatically better.

4. Independent tasks can potentially run in parallel.
5. Dependent tasks usually need sequential execution.
6. Use specialized agents when responsibilities naturally differ.
7. Use human approval for high-impact or risky actions.
8. Do not retry every failure blindly.
9. State represents current task progress; it does not necessarily mean permanent memory.
10. Agentic behavior comes from dynamically deciding actions based on goals, observations, tools, and constraints.

23. Day-1 Learning Status

Topic	Status
AI Agent	First-pass understood
Agent vs Workflow	First-pass understood
Agent Loop	First-pass understood
Tools / Capabilities	First-pass understood
Autonomy	First-pass understood
Roles & Specialization	First-pass understood
Handoffs	First-pass understood
Supervisor / Manager	First-pass understood
Single vs Multi-Agent	First-pass understood
Task Decomposition	First-pass understood
Sequential vs Parallel	First-pass understood
Planning	First-pass understood
State	First-pass understood
Human-in-the-Loop	First-pass understood
Failure Handling / Retries	First-pass understood
Orchestration	First-pass understood
Scenario Practice	Next
EOD Revision	Pending
Day-1 Assessment	Pending

24. Exam Thinking Framework

1. What is the goal?
2. Is the task simple or complex?
3. Does the task need dynamic decision-making?
4. What tools/capabilities are required?
5. How much autonomy is appropriate?

6. Can tasks run independently?
7. Does one task depend on another?
8. Does specialization provide real value?
9. Is human approval required?
10. What happens when something fails?

Current position: First-pass understanding is around 7/10. The next stage is applying these ideas to architecture scenarios, then doing an end-of-day revision and assessment.